

TRABAJO FINAL INTEGRADOR · ENTREGA 01 — PROPUESTA DE PROYECTO Y REPOSITORIO

La Comanda

Sistema web de gestión de menú y pedidos
para comercios gastronómicos

Equipo	Stefan Dios Mayarin · Mathias Jesus Flor
Tutora académica	María Candela Grosso
Carrera	Tecnicatura Universitaria en Programación a Distancia (TUPaD) — UTN
Stack tecnológico	React + TypeScript · Node.js/Express · MongoDB Atlas

Contenido

1. Resumen ejecutivo.....	3
2. Identificación del problema.....	3
3. Propuesta de solución.....	5
4. Alcance	6
5. Stack tecnológico y justificación	7
6. Análisis de mercado y diferenciación	8
7. Plan de trabajo.....	8
8. Uso de inteligencia artificial en el proyecto	9

1. Resumen ejecutivo

El presente documento constituye la propuesta formal del Trabajo Final Integrador del equipo. Proponemos desarrollar **La Comanda**, un sistema web de gestión de menú y pedidos orientado a comercios gastronómicos pequeños y medianos. La solución reemplaza la carta estática (impresa, pizarra o PDF reenviado por WhatsApp) y la toma manual de pedidos por una plataforma centralizada: el cliente consulta una carta siempre vigente y arma su pedido; el administrador gestiona platos, precios y pedidos en tiempo real y obtiene un registro histórico que antes no existía.

La propuesta define un MVP acotado y desplegado en producción, un stack tecnológico justificado y un plan de trabajo con metodología y objetivos medibles, conforme a los lineamientos de la primera entrega del Trabajo Final Integrador.

Ficha técnica	
Proyecto	La Comanda — sistema web de gestión de menú y pedidos
Tipo de solución	Aplicación web responsive: SPA (frontend) + API REST (backend)
Stack	React + TypeScript · Tailwind CSS · Node.js + Express · MongoDB Atlas · JWT
Despliegue	Netlify (frontend) · Render (backend) · MongoDB Atlas — costo de operación: \$0
Metodología	Scrum adaptado + tablero Trello · Git y GitHub (repositorio único con carpetas frontend y backend)
Repositorio	https://github.com/stefanmdev/la-comanda — repositorio único del proyecto.

2. Identificación del problema

2.1 Contexto

En los comercios gastronómicos pequeños y medianos —restaurantes de barrio, rotiserías, cafeterías— la carta suele vivir en soportes estáticos: menús impresos o plastificados, pizarras, o un PDF/imagen que se reenvía por WhatsApp. Los pedidos se toman a mano en papel, por teléfono o en el mostrador, y se transcriben verbalmente a la cocina. En un contexto donde los precios se actualizan con mucha frecuencia, mantener la carta al día implica reimprimir o reenviar archivos, y es habitual que el cliente vea un precio y termine pagando otro.

El proceso «funciona», pero constituye lo que el material del módulo denomina una **ineficiencia tolerada**: se sostiene por costumbre, no porque sea eficiente. El negocio no percibe el costo acumulado de los errores, las demoras y la información que se pierde en cada jornada.

2.2 Actores y necesidades

Actor	Rol en el proceso	Necesidades principales
Cliente (comensal)	Consulta la carta y realiza pedidos	Ver precios y disponibilidad reales; pedir sin fricción ni esperas; conocer el estado de su pedido
Dueño / administrador	Define la carta y supervisa la operación	Actualizar platos y precios al instante; ver los pedidos ordenados y su estado; contar con registro histórico para decidir

Actor	Rol en el proceso	Necesidades principales
Personal de salón / cocina	Toma, prepara y entrega los pedidos	Recibir pedidos claros, completos y sin errores de transcripción, incluso en hora pico

2.3 Problema central

Los comercios gastronómicos pequeños y medianos que gestionan su carta en soportes estáticos y toman pedidos de forma manual enfrentan tres problemas encadenados: información desactualizada (precios y disponibilidad), errores y demoras en la toma de pedidos, y ausencia total de registro digital sobre qué se vende, cuánto y cuándo. El impacto lo pagan el cliente (confusiones, esperas, mala experiencia), el personal (retrabajo y reclamos) y el negocio (tiempo, dinero y decisiones a ciegas).

2.4 Impacto

- **Tiempo y dinero:** cada actualización de precios obliga a reimprimir cartas o reenviar archivos; con un sistema centralizado el cambio tarda segundos y no tiene costo.
- **Errores:** los pedidos anotados a mano se transcriben mal o se olvidan; cada error implica retrabajo en cocina, demora en la mesa y un cliente insatisfecho.
- **Información perdida:** sin registro digital no existe historial de pedidos ni estadísticas (platos más vendidos, franjas de mayor demanda); las decisiones sobre la carta se toman por intuición.
- **Experiencia degradada:** cartas vencidas, precios que «no eran» y esperas en hora pico deterioran la experiencia y la reputación del local (reseñas y recomendaciones).

2.5 Valor agregado real

Siguiendo el criterio del módulo, la solución no vale por digitalizar sino por lo que habilita. Un menú en PDF ya es «digital» y no resuelve nada; el valor de La Comanda está en la centralización, la actualización instantánea y el registro:

- **Reduce costos:** actualizar un precio pasa de horas (reimpresión) a segundos; disminuyen los errores de toma de pedido y el esfuerzo del personal en hora pico.
- **Hace posible lo que antes era imposible:** cada pedido queda registrado con fecha, ítems y estado; eso habilita historial y estadísticas que con papel, pizarra o PDF directamente no existen.
- **Mejora la experiencia de forma medible:** carta siempre vigente, pedido autogestionado desde la mesa y estado del pedido visible tanto para el cliente como para el administrador.

2.6 Validación del problema

El problema es **real** (ocurre hoy en la enorme mayoría de los comercios chicos del rubro), **relevante** (afecta ingresos, tiempos y reputación) y **remediable** (existe tecnología madura y accesible para resolverlo). Existen soluciones parciales —menú QR estático, pedidos por WhatsApp— que no alcanzan: no actualizan disponibilidad, no registran pedidos de forma estructurada ni ofrecen un panel de gestión. Como validación adicional, durante la primera semana del proyecto contrastaremos estos supuestos con

un mini-relevamiento (entrevista breve) a un comerciante gastronómico de nuestro entorno.

3. Propuesta de solución

3.1 Descripción general

La **Comanda** es una aplicación web responsive con dos caras. Por un lado, una **carta digital pública** donde cualquier visitante consulta la información del local y los platos disponibles, organizados por categoría, con precios vigentes. Por el otro, un **flujo autenticado**: el cliente registrado arma su carrito, confirma el pedido indicando su número de mesa y sigue su estado; el administrador gestiona la carta, los pedidos y los usuarios desde un panel. Toda la información vive en una única base de datos, por lo que cada cambio del administrador se refleja al instante en lo que ve el cliente. El nombre retoma la «comanda», el papel con el que históricamente viaja el pedido a la cocina: exactamente el proceso que el sistema digitaliza y potencia.

3.2 Roles del sistema

Rol	Capacidades
Visitante	Navegar la home y consultar la carta completa con precios y disponibilidad
Cliente (registrado)	Todo lo anterior + armar carrito, confirmar pedidos con número de mesa y consultar el historial y estado de sus pedidos
Administrador	Gestión completa: CRUD de platos, cambio de estado de pedidos, listado de usuarios y de pedidos

3.3 Módulos principales

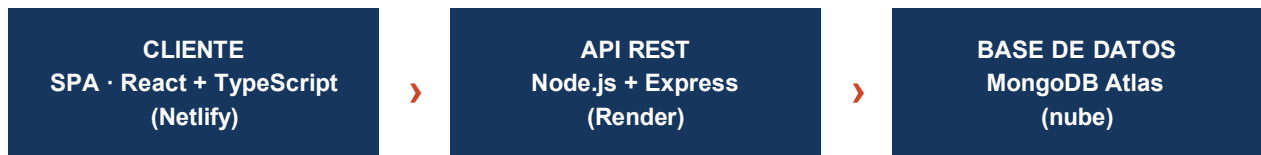
- **Autenticación y autorización:** registro e inicio de sesión con email y contraseña, tokens JWT, rutas protegidas por rol en el frontend (React Router) y en la API.
- **Catálogo (carta):** platos con nombre, detalle, precio, categoría, imagen y estado; vista pública organizada por categorías.
- **Pedidos:** carrito con cantidades y total calculado; confirmación que persiste el pedido como «pendiente»; seguimiento de estados.
- **Panel de administración:** ABM de platos, gestión de pedidos (pendiente › realizado / cancelado) y listados de usuarios y pedidos.
- **API REST documentada:** endpoints probados y publicados como colección de Postman.

3.4 Modelo de datos (colecciones)

Colección	Campos principales
usuarios	nombre, email (único), password (hash bcrypt), rol [cliente admin], estado, fecha de alta
platos	nombre, detalle, precio, categoría, imagen (URL), estado [activo inactivo], stock simple (deseable, etapa 2)
pedidos	usuario (ref.), fecha, ítems [plato ref., nombre, precio unitario, cantidad], total, número de mesa, estado [pendiente realizado cancelado]
categorías (etapa 2)	nombre, estado — en el MVP la categoría es un campo del plato

Nota de diseño: cada ítem del pedido guarda una copia del nombre y el precio del plato al momento de pedir. Así, los cambios futuros de precios no alteran el historial: los pedidos viejos siguen mostrando lo que realmente se cobró.

3.5 Arquitectura



El frontend consume la API mediante HTTPS con intercambio JSON; las rutas privadas envían el token JWT en el encabezado Authorization. Todo el proyecto vive en un **único repositorio de GitHub** —conforme a los lineamientos de la asignatura— con carpetas separadas **frontend/** y **backend/**, y cada componente se despliega de forma independiente con deploy continuo desde ese repositorio: Netlify toma la carpeta del frontend y Render la del backend.

4. Alcance

4.1 MVP (producto mínimo viable)

- 1. Registro e inicio de sesión con email y contraseña (hash con bcrypt) y autenticación con JWT.
- 2. Roles Cliente y Administrador, con rutas protegidas en el frontend y autorización en la API.
- 3. Home pública con información del local y la carta organizada por categorías, con precios y disponibilidad actualizados.
- 4. Carrito: selección de platos, cantidades y total calculado.
- 5. Confirmación de pedido con número de mesa; el pedido se persiste con estado «pendiente».
- 6. «Mis pedidos»: historial y estado de los pedidos del cliente autenticado.
- 7. Panel admin: CRUD completo de platos (alta, baja, modificación y listado).
- 8. Gestión de pedidos por el admin: listado y cambio de estado (pendiente › realizado / cancelado).
- 9. Listado de usuarios registrados (admin).
- 10. Diseño responsive (mobile first), API documentada con colección Postman y ambos componentes desplegados en producción (Netlify + Render) con datos en MongoDB Atlas.

4.2 Deseable si el tiempo lo permite (nice to have)

Esta lista incorpora los cinco puntos «bonus» del proyecto de referencia y dos extras definidos por el equipo (stock simple y pago simulado). Se abordarán recién con el MVP cerrado y en este orden de prioridad, elegido por su relación valor demostrable / esfuerzo:

- 1. **Búsqueda y filtrado de platos por categoría, con paginación del listado** (*bonus*).
- 2. **Stock simple por plato**: cantidad disponible que se descuenta al confirmarse cada pedido; el plato sin stock se muestra como «agotado» y no puede agregarse al carrito. Es distinto del inventario de insumos (fuera de alcance): un único campo numérico, mucho valor en la demo.
- 3. **Página de pago simulada** (*bonus*): checkout propio que valida los datos y marca el pedido como «pagado», sin depender de servicios externos — siempre funciona en la defensa. Como extensión opcional, integración con Mercado Pago en **modo de prueba (sandbox)**, solo si el resto está cerrado y sobra tiempo.
- 4. **Email de bienvenida** al registrarse (*bonus*).
- 5. **CRUD de usuarios con aprobación** por el administrador (*bonus*).
- 6. CRUD de categorías como colección propia.
- 7. Estado adicional «en preparación» y notificación visual de cambios de estado.

- **8.** Carga de imágenes a un servicio externo (p. ej. Cloudinary) en lugar de URL.
- **9.** Mini-estadísticas para el admin: platos más pedidos y pedidos por día.
- **10.** Acceso a la carta mediante código QR por mesa.

4.3 Explícitamente fuera de alcance

Para proteger la viabilidad temporal, la primera versión **no** incluirá: pagos con dinero real (cualquier pasarela se contempla únicamente en modo de prueba, como extensión opcional del punto 3 anterior), facturación electrónica/fiscal, gestión de inventario de insumos e ingredientes (distinto del stock simple por plato, que sí es deseable), gestión de delivery y logística, aplicaciones móviles nativas, notificaciones en tiempo real (websockets / push) ni soporte multi-sucursal. Estas ideas quedan registradas en el backlog como una eventual segunda versión.

5. Stack tecnológico y justificación

5.1 Componentes elegidos

Capa	Elección	Justificación breve
Frontend	React 18 + TypeScript (Vite)	Componentes reutilizables; TypeScript agrega tipado que reduce errores de integración al trabajar de a dos
Ruteo	React Router	Estándar del ecosistema para rutas públicas y protegidas por rol
Estilos	Tailwind CSS + CSS propio	Velocidad de maquetado responsive y consistencia visual sin sobrecargar el proyecto
Backend	Node.js + Express	Unifica el lenguaje (JavaScript/TypeScript) en todo el stack; ideal para una API REST transaccional liviana; experiencia previa del equipo
Base de datos	MongoDB Atlas + Mongoose	El modelo de documentos es natural para la carta y para pedidos con ítems embebidos; tier gratuito estable
Autenticación	JWT + bcrypt	Estándar sin estado para SPA + API; contraseñas nunca en texto plano
Pruebas de API	Postman	Colección documentada
Despliegue	Netlify (front) · Render (API)	PaaS gratuitos con deploy continuo desde GitHub y HTTPS incluido; restricción económica: costo \$0
Versionado	Git + GitHub (repositorio único: frontend/ + backend/)	Requisito de la asignatura; trabajo en paralelo con ramas por funcionalidad y pull requests con revisión cruzada
Gestión	Trello + Scrum adaptado	Sprints semanales y tablero visible para el equipo
Asistencia de IA	Copilot / ChatGPT / Claude	Refactorización, unit testing y revisión, siempre con validación humana (ver sección 10)

5.2 Escala esperada

El escenario real analizado es el de un único local con decenas de usuarios simultáneos como techo realista, más el uso académico de la demo. Node.js, Express y Atlas cubren ese escenario con margen amplio. Evitamos deliberadamente la sobreingeniería (microservicios, contenedores orquestados): sería

impresionante pero injustificado, tal como advierte la lectura del módulo. Si el sistema creciera, el camino natural es escalar el plan de Atlas y las instancias del PaaS sin cambios de arquitectura.

6. Análisis de mercado y diferenciación

6.1 Competencia

Competidores directos: sistemas gastronómicos comerciales con suscripción mensual (p. ej. Fudo, Maxirest), plataformas SaaS de carta digital / menú QR con abono, y aplicaciones de pedidos con comisión por transacción orientadas a delivery (p. ej. PedidosYa). **Competidores indirectos:** el statu quo — menú PDF o imagen por WhatsApp, pizarras, redes sociales y anotadores de papel; compiten porque son gratis y conocidos, aunque no resuelvan el problema.

6.2 Diferenciación

- **Sin abono ni comisiones:** solución a medida con costo de operación cero, frente a suscripciones mensuales o comisiones por pedido.
- **Foco en el salón y el mostrador** (número de mesa, estados simples), no en el delivery.
- **Simplicidad radical:** pensada para un comercio chico sin personal técnico; menos funciones, mejor hechas.
- **Propiedad total** del código y de los datos, con base extensible: estadísticas, QR por mesa y una futura app móvil (React Native) como evolución natural.

6.3 Análisis FODA

FORTALEZAS • Stack dominado por el equipo • Alcance acotado y validado como proyecto formativo • Requerimientos claros desde el día uno	OPORTUNIDADES • Demanda real de digitalización gastronómica (precios que cambian seguido) • Base extensible: estadísticas, QR, app móvil • Pieza de portfolio profesional para ambos integrantes
DEBILIDADES • Equipo de dos con dedicación parcial • Sin cliente contratante real en esta etapa • Primera experiencia desplegando un producto completo de punta a punta	AMENAZAS • Cambios en las condiciones de los servicios gratuitos • Abundancia de soluciones comerciales existentes • Picos de carga académica y laboral durante el cuatrimestre

7. Plan de trabajo

7.1 Metodología

Trabajaremos con **Scrum adaptado a un equipo de dos**: sprints semanales con planning breve los lunes, seguimiento diario asincrónico por chat y review + retrospectiva al cierre de cada sprint. El tablero **Trello** refleja el estado real del proyecto. En GitHub trabajamos con ramas por funcionalidad (feature/*) integradas a una rama de desarrollo mediante pull requests con revisión cruzada obligatoria, y commits bajo la convención Conventional Commits (feat:, fix:, docs:, chore:).

7.2 Objetivos

Objetivo general: desarrollar, desplegar y defender el MVP completo de La Comanda, con documentación y trazabilidad de todo el proceso.

- **OE1.** API REST con autenticación JWT, roles y CRUDs, desplegada en producción y documentada con una colección Postman completa.
- **OE2.** SPA responsive desplegada, con carta pública, flujo de pedido y panel de administración.
- **OE3.** Flujo completo registro › pedido › gestión demostrable en producción sin errores críticos.
- **OE4.** Repositorio único con historial de commits equilibrado de ambos integrantes y tablero al día.

8. Uso de inteligencia artificial en el proyecto

En esta etapa de propuesta, utilizamos IA como interlocutor técnico, siguiendo la Actividad 3: para confrontar y poner bajo estrés las dos ideas candidatas, comparar alternativas con criterios explícitos, detectar casos borde y puntos ciegos (historial de precios, estados de pedido, riesgos de los tiers gratuitos) y estructurar el plan de trabajo. Las decisiones finales —el proyecto elegido, el stack y el alcance— fueron tomadas y son defendidas por el equipo.

Durante el desarrollo, usaremos asistentes (GitHub Copilot, ChatGPT, Claude) para refactorización, generación de pruebas unitarias, revisión de código y consultas puntuales, conforme a los lineamientos de la incubadora. Regla del equipo: ningún código asistido se integra sin ser leído, probado y comprendido por quien lo integra; el criterio arquitectónico y la lógica de negocio son responsabilidad exclusiva de los estudiantes, y la defensa técnica se realiza sin asistencia.